VitalWatch

See the trend before the threshold.

TRACK
AI for Real-Time Decision Making

FORMAT
Hackathon idea proposal and technical plan

PROTOTYPE
FastAPI · HTML/CSS/JavaScript · ML · Historical hourly replay

A transparent, replayable early-warning research prototype that combines multivariate vital-sign trends, missingness signals, and an interpretable ML model to prioritize clinician assessment.

> *Safety boundary: alerts prioritize review; they do not diagnose sepsis or recommend treatment.*

2. Problem Statement

The core issue

Patient deterioration often develops as a pattern rather than a single dramatic measurement. Heart rate may gradually rise while respiratory rate increases, oxygen saturation declines, blood pressure weakens, and temperature changes. Each individual value may remain below a conventional alert threshold, while the combined trajectory is concerning.

A threshold-only workflow can therefore produce two problems:

- **Late recognition:** an alert appears only after a vital sign crosses a fixed limit.
- **Alert overload:** isolated threshold breaches generate noisy notifications without enough context.

VitalWatch addresses the gap by analyzing recent changes across several measurements and updating a patient-level risk score as new observations become available.

Target audience

The primary users are:

- Clinicians and nursing teams who need to prioritize patient review.
- Clinical informatics and quality-improvement teams exploring early-warning workflows.
- Researchers and datathon participants building transparent deterioration models.
- Health-tech teams evaluating how historical clinical data can support real-time-style decision tools.

The prototype is not intended to make autonomous clinical decisions. It surfaces a ranked worklist and supporting evidence for a qualified human reviewer.

Why now?

Clinical data is increasingly timestamped and available as a sequence of observations, but many demonstrations still focus on static dashboards or isolated threshold rules. There is an opportunity to combine:

- Causal rolling trends rather than future-aware summaries.
- Missingness indicators, because measurement frequency can reflect clinical workflow.
- Patient-level machine-learning evaluation rather than row-level random splits.
- A replayable interface that demonstrates how risk changes over time.

The PhysioNet/CinC Challenge 2019 dataset provides a practical starting point for a narrowly defined sepsis early-warning task. Its records are hourly rather than second-by-second bedside waveforms, so VitalWatch will replay them honestly as historical hourly observations.

3. Proposed Solution Overview

The pitch

VitalWatch is an AI-assisted clinical prioritization dashboard. It receives a historical or synthetic patient record one observation at a time, calculates recent multivariate trends, estimates the risk of a sepsis-related deterioration event within a defined future window, and presents the most concerning patients first with a concise evidence panel.

No IoT device is required for the prototype. A FastAPI replay service releases historical or synthetic rows chronologically so the dashboard behaves like a live system while preserving the data's original hourly resolution.

Key features

- **Patient priority board:** Rank patients by current model risk and risk change.
- **Chronological replay:** Play, pause, step forward, reset, and change playback speed.
- **Multivariate risk scoring:** Combine current values, rolling summaries, trends, and missingness indicators.
- **Threshold comparison:** Show how a simple threshold baseline compares with the learned model.
- **Risk history:** Plot the patient's risk score over time with alert and retrospective onset markers.
- **Vital-sign trends:** Display heart rate, respiratory rate, oxygen saturation, blood pressure, and temperature.
- **Missing-data visibility:** Show missing measurements explicitly rather than treating them as normal values.
- **Alert explanations:** Describe model-associated evidence such as rising respiratory rate or declining oxygen saturation.
- **Simulation labeling:** Clearly mark the interface as historical replay or synthetic demonstration mode.
- **Evaluation view:** Report lead time, sensitivity at a selected false-alert rate, and patient-level performance.

Value proposition

VitalWatch is valuable because it combines a credible ML workflow with a visible, replayable decision interface:

- It detects **patterns and trajectories**, not only isolated threshold breaches.
- It keeps the human decision-maker in the loop.
- It makes the model's current evidence visible rather than presenting an unexplained score.
- It demonstrates real-time-style behavior without requiring sensors, hospital integration, or protected live data.
- It is lightweight enough to build during a hackathon and extensible toward future streaming inputs.

Intended AI contribution

The AI component is a supervised early-warning model. At time `t`, it uses only observations available through `t` and predicts whether the patient will receive a sepsis label within a predefined future horizon, such as the next six hours.

The minimum model is an interpretable logistic regression. A fixed threshold rule serves as the baseline. The dashboard compares their alert timing and false-alert burden.

4. Architecture & Technical Methodology

Tech stack

Layer	Proposed technology	Purpose
Frontend	HTML, CSS, vanilla JavaScript	Custom, aesthetic clinical command-center interface
Charts	Apache ECharts or Chart.js	Responsive multi-series vital and risk charts
Backend	Python, FastAPI, Uvicorn	REST endpoints, replay control, prediction service
Live updates	WebSocket	Push each simulated observation and risk update to the browser
Data processing	Pandas, NumPy	Normalize records and create causal rolling features
ML	scikit-learn	Logistic regression, preprocessing, metrics, serialization
Storage	CSV/JSON for demo; SQLite or DuckDB for persistence	Synthetic data, predictions, replay state, evaluation results
Model artifact	Joblib	Save and load the trained prototype model
Deployment	Local server or container	Reproducible hackathon demonstration

FastAPI is preferred over Django for the MVP because the core workflow is an API, replay engine, and WebSocket stream. Django becomes attractive later if the system requires accounts, role-based permissions, an administrative console, audit logs, or multi-tenant workflows.

Data sources and data strategy

Model development and evaluation

Use the **PhysioNet/CinC Challenge 2019** dataset as the primary real-world starting point. It provides patient-hour records, vital signs, laboratory values, and a `SepsisLabel` outcome. The MVP can focus on:

- Heart rate (`HR`)
- Oxygen saturation (`O2Sat`)
- Temperature (`Temp`)
- Systolic blood pressure (`SBP`)
- Diastolic blood pressure (`DBP`)
- Respiratory rate (`Resp`)

Dashboard replay

Use a small synthetic CSV or JSON file containing stable, worsening, noisy, and missing-data patient trajectories. The synthetic rows are released chronologically by the replay engine. They are for interface demonstration and pipeline testing, not clinical validation.

Required row schema

```
patient_id
timestamp
heart_rate
respiratory_rate
oxygen_saturation
systolic_bp
diastolic_bp
temperature
heart_rate_missing
respiratory_rate_missing
oxygen_saturation_missing
blood_pressure_missing
temperature_missing
sepsis_label                # retrospective evaluation only
```

The model must not receive the future `sepsis_label` during replay. Blank values and explicit missingness indicators should be retained.

ML methodology

Target definition

For a prediction time `t`:

```
Input: all measurements available through time t
Target: whether the first sepsis label occurs within the next six hours
```

The prototype can define sepsis onset operationally as the first timestamp at which `SepsisLabel` equals 1. This is a dataset-based definition and must not be presented as the exact biological onset time.

Causal features

Features are calculated from the current and prior observations only:

- Current vital-sign values.
- Three-, six-, and twelve-hour rolling means, minima, maxima, and variability.
- Linear trend or slope for each vital sign.
- Change from a recent patient-specific baseline calculated using past data only.
- Hours since the last available measurement.
- Count of missing vital signs in the current window.
- Persistence of increasing or decreasing trends.

Models

1. **Threshold baseline:** A transparent rule that alerts when multiple vital signs cross prototype thresholds.
2. **Primary ML model:** Logistic regression with imputation, scaling, rolling features, and missingness indicators.
3. **Optional comparison:** A gradient-boosted tree model if time permits.

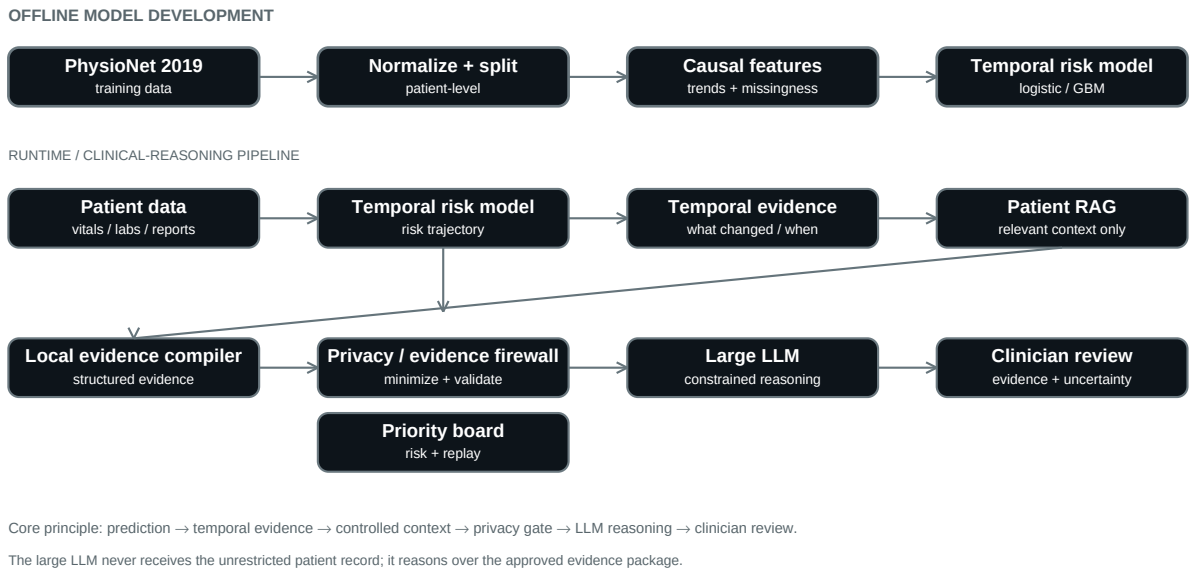
The first version should favor an interpretable model over a complex neural network. The project does not need an LLM or chatbot to qualify as AI; the learned sequential risk estimate is the core AI contribution.

Leakage controls

- Split data by patient, never by individual hourly rows.
- Keep all records from a patient in exactly one of train, validation, or test sets.
- Fit imputation, scaling, feature selection, and calibration on training data only.
- Calculate every rolling feature using only rows available at that prediction time.

- Do not use future labels, future vitals, or full-stay patient averages.
- Tune the alert threshold on validation patients and report final performance on held-out test patients.

System architecture / flow diagram



Core principle: prediction → temporal evidence → controlled context → privacy gate → LLM reasoning → clinician review.

The large LLM never receives the unrestricted patient record; it reasons over the approved evidence package.

Suggested API surface

```
GET  /                          Dashboard page
GET  /api/patients              Patient priority data
GET  /api/patients/{id}         Current patient state
GET  /api/patients/{id}/history Risk and vital history
POST /api/replay/start          Start playback
POST /api/replay/pause          Pause playback
POST /api/replay/step           Advance one simulated hour
POST /api/replay/reset          Reset replay state
WS   /ws/replay                 Stream observations and predictions
```

Feasibility

The prototype is feasible within a hackathon because:

- The primary dataset and feature schema are well defined.
- The ML model can be implemented with standard scikit-learn components.
- The replay engine can use a CSV rather than requiring device integration.
- FastAPI provides REST and WebSocket support with little infrastructure.
- HTML/CSS/JavaScript allows a polished interface without adopting a large frontend framework.
- The first model can be trained offline and loaded by the API.
- The scope is intentionally narrow: sepsis-related early warning rather than general diagnosis.

5. Impact, Scalability & Roadmap

Impact metrics

The prototype should report both model performance and operational usefulness:

- **Lead time:** Time between the first sustained model alert and the retrospective labelled onset.
- **Sensitivity:** Percentage of sepsis-positive patients alerted before onset.
- **False alerts per patient-day:** Alert burden at a selected operating threshold.
- **Sensitivity at fixed false-alert rate:** Compare the ML model with the threshold baseline.
- **Patient-level AUROC and AUPRC:** Report performance on held-out patients.
- **Calibration:** Assess whether risk scores behave consistently as probability-like outputs.
- **Coverage of explanations:** Percentage of alerts with a readable evidence summary.
- **Replay latency:** Time from receiving a row to displaying the updated risk.

The primary demonstration should show one patient whose risk rises before the labelled onset while a stable patient remains low priority.

Scalability beyond the prototype

The architecture can evolve in stages:

1. Replace CSV replay with an authenticated EHR, HL7/FHIR, message-queue, or device gateway.
2. Replace in-memory replay state with durable storage and audit logging.
3. Add user authentication, role-based access, and organization-level data isolation.
4. Add model monitoring, drift detection, data-quality checks, and alert acknowledgement workflows.
5. Validate on external hospitals and different patient populations.
6. Add prospective silent-mode evaluation before any clinical workflow integration.
7. Support additional deterioration outcomes only after separate label and validation work.

MVP roadmap

Phase 1 — Project setup and data contract

- Confirm target definition and six-hour prediction horizon.
- Download and inspect the PhysioNet dataset.
- Normalize the six vital-sign fields and missingness indicators.
- Define the synthetic replay schema.

Phase 2 — Feature and model pipeline

- Build causal rolling features.
- Implement the threshold baseline.
- Train logistic regression with a patient-level split.
- Save the model and calculate validation metrics.

Phase 3 — Backend replay service

- Implement FastAPI endpoints.
- Load synthetic demo cases.

- Add step, pause, play, speed, and reset controls.
- Broadcast patient updates over WebSocket.

Phase 4 — Frontend dashboard

- Build the clinical command-center layout.
- Add patient priority cards and vital-sign cards.
- Add risk and vital-sign charts.
- Add alert markers, evidence text, and missing-data indicators.
- Add an always-visible simulation disclaimer.

Phase 5 — Evidence and final demo

- Replay one worsening and one stable patient.
- Compare model alerts with the threshold baseline.
- Calculate lead time and patient-level results.
- Prepare limitations and validation notes.
- Package the code, model artifact, synthetic data, and PDF proposal.

Suggested hackathon schedule

Time	Deliverable
Hours 0–4	Scope, data contract, repository setup, synthetic cases
Hours 4–10	Data normalization and causal feature pipeline
Hours 10–16	Baseline, logistic regression, patient-level evaluation
Hours 16–24	FastAPI endpoints and replay WebSocket
Hours 24–34	Frontend dashboard, charts, and visual polish
Hours 34–40	Connect model, tune replay, write explanations
Hours 40–48	Test, capture evidence, rehearse demo, finalize documentation

6. References & Appendix

Research and datasets

1. **PhysioNet/CinC Challenge 2019: Early Prediction of Sepsis from Clinical Data**

Dataset and challenge information: https://physionet.org/content/challenge-2019/1.0.0/

2. **PhysioNet**

Open research resources for physiological and clinical data: https://physionet.org/

3. **Reyna, M. A. et al.** Early Prediction of Sepsis from Clinical Data: The PhysioNet/Computing in Cardiology Challenge 2019. The challenge publication and citation information are available through the PhysioNet Challenge 2019 resource page.

Technical documentation

4. **FastAPI documentation**

https://fastapi.tiangolo.com/

5. **FastAPI WebSockets**

https://fastapi.tiangolo.com/advanced/websockets/

6. **scikit-learn documentation**

https://scikit-learn.org/stable/

7. **Apache ECharts documentation**

https://echarts.apache.org/en/index.html

Appendix A — Example synthetic patient trajectory

```
Patient: P001
Mode: Synthetic demonstration
Resolution: One observation per simulated hour

08:00  HR 82   RR 17  SpO2 98  BP 122/76  Temp 36.8  Risk 0.08
09:00  HR 85   RR 18  SpO2 97  BP 119/74  Temp 37.0  Risk 0.12
10:00  HR 89   RR 20  SpO2 96  BP 114/71  Temp 37.3  Risk 0.19
11:00  HR 94   RR 22  SpO2 95  BP 108/68  Temp 37.7  Risk 0.31
12:00  HR 99   RR 24  SpO2 94  BP 102/64  Temp --    Risk 0.44
13:00  HR 104  RR 26  SpO2 92  BP 96/60   Temp 38.4  Risk 0.67 ALERT
14:00  HR 108  RR 29  SpO2 91  BP 91/57   Temp 38.7  Risk 0.81 LABELLED ONSET
```

Appendix B — Demonstration language

Use the following wording in the dashboard and presentation:

> *“VitalWatch is a retrospective hourly-data replay demonstrating sequential risk updating. Alerts prioritize clinician assessment; they do not diagnose sepsis or recommend treatment.”*

> *“The explanation lists model-associated evidence, not proven clinical causes.”*

> *“A datathon model supports a research demonstration. Clinical deployment requires prospective, external, and workflow-specific validation.”*

Appendix C — Scope boundaries

The MVP deliberately does not claim:

- Second-by-second waveform monitoring.
- Continuous sensor integration.
- Clinical diagnosis.
- Treatment recommendations.
- Autonomous escalation.
- Generalization to all hospitals or patient groups.
- Prospective clinical benefit.

Appendix D — New AI Idea & Research Framing

The original temporal early-warning system remains the core predictive component. The new idea adds a controlled reasoning layer on top of it.

Updated one-line project statement

A system that combines temporal patient data with privacy-preserving LLM reasoning to detect deterioration and explain what might be happening.

What is new

- Temporal risk model: **Is the patient deteriorating?**
- Temporal evidence engine: **What changed, when, how fast, and how persistent?**
- Patient RAG: **What relevant historical context is needed?**
- Local evidence compiler: retrieved chunks → minimal structured evidence.
- Privacy / evidence firewall: validation, identifier minimization, leakage and prompt-injection checks.
- Large LLM: **What plausible explanations should a clinician consider, given this evidence?**

Correct framing

The LLM is not the primary detector and is not an autonomous diagnostic system. It is the **reasoning and explanation layer constrained by the temporal model**.

Research question

Does constraining an LLM with temporal physiological evidence and selectively retrieved patient context produce more reliable, explainable and privacy-conscious clinical reasoning than asking an LLM to reason directly from patient data?

This extends the original temporal deterioration idea rather than replacing it.

Appendix E — Privacy-Preserving Patient RAG

The RAG layer is not an unrestricted patient-memory system. Retrieval is conditioned by the deterioration event or reasoning task identified by the temporal system.

Stage	Role
Private patient data	Vitals, labs, reports, medications and prior history remain inside the controlled patient-data boundary.
Temporal evidence engine	Defines the current deterioration pattern and reasoning context.
Relevance-bounded Patient RAG	Retrieves only information relevant to the current event/task, with provenance.
Local evidence compiler	Converts raw retrieved chunks into strict structured evidence rather than forwarding raw text.
Privacy / evidence firewall	Identifier minimization, schema validation, leakage checks, prompt-injection checks and provenance.
Large LLM	Receives only the approved evidence package and performs constrained reasoning.
Clinician review	Shows differential possibilities, evidence, uncertainty and traceability.

Example

If the temporal system detects respiratory deterioration, retrieval may request recent respiratory reports, relevant medications and prior respiratory history. Unrelated history should not be retrieved simply because it exists.

Structured evidence passed to the LLM

Field	Example
Temporal event	Respiratory deterioration pattern
Observed change	SpO2 falling while respiratory rate rises
Persistence / baseline	Pattern persists and increasingly deviates from recent baseline
Relevant context	Only retrieved information necessary for interpretation
Contradictions / missing data	Evidence against an interpretation and unavailable context
Provenance	Where the item came from and why it was retrieved

Privacy evaluation

- Direct identifier extraction and indirect leakage tests.
- Prompt-injection tests on retrieved clinical notes.
- Minimum-disclosure comparison.
- Do not claim perfect anonymity; use privacy-preserving / privacy-conscious language.

Appendix F — Constrained LLM Reasoning & Updated Roadmap

The large LLM receives a structured evidence package instead of the raw patient chart. Its output is constrained so reasoning remains traceable and uncertainty is explicit.

LLM output contract

Output	Requirement
Differential possibilities	Plausible explanations, not confirmed diagnoses.
Supporting evidence	Major claims linked to supplied time-stamped evidence.
Evidence against	Contradictory or weakening evidence.
Temporal reasoning	Direction, persistence and timing.
Missing information	What would reduce uncertainty.
Uncertainty	Speculation clearly separated from evidence.
Evidence trace	Reviewer can inspect source evidence.

Experimental comparison

Snapshot LLM	Temporal LLM	Proposed system
Current measurements → LLM	Current + history → LLM	Temporal model → evidence → selective RAG → local compiler → firewall → LLM

Updated roadmap

- **Data/model:** retain PhysioNet 2019, patient-level splits, causal features and temporal ML.
- **Backend:** retain FastAPI replay/prediction service and WebSocket stream.
- **Frontend:** retain historical replay and priority board; add temporal-evidence view.
- **Reasoning:** add event-conditioned Patient RAG, local evidence compilation and provenance.
- **Privacy:** add evidence firewall and leakage / prompt-injection evaluation.
- **LLM:** add constrained differential reasoning with evidence, uncertainty and traceability.

Final safety framing

Alerts prioritize review. The LLM generates plausible differential insights from supplied evidence; it does not independently diagnose, prescribe treatment or escalate care. This remains a research prototype.

Architecture principle

Prediction → evidence → controlled context → privacy gate → reasoning → clinician review.

Appendix G - Future Development

SMARTPHONE-BASED ACTIVE HEALTH MONITORING | NOT PART OF VITALWATCH v1.0

Purpose. A later version of VitalWatch may accept selected, patient-initiated measurements captured with a smartphone. These observations would complement medical reports, not replace them. The current MVP remains the historical/synthetic early-warning replay and its temporal evidence, Patient RAG, privacy/evidence firewall and constrained LLM pipeline.

Planned smartphone capabilities

1. Camera-based resting heart-rate estimate

A guided fingertip-on-camera-and-flash session uses photoplethysmography (PPG) to estimate pulse rate. The app records the estimate, timestamp, device information and signal-quality result. Raw video should be processed on-device and discarded by default.

Output: Resting heart-rate estimate (BPM) plus quality metadata. This is not an ECG, blood-pressure or arrhythmia measurement.

2. Guided mobility assessment

A later app flow may guide a five-times sit-to-stand assessment. On-device video pose estimation or motion sensors can estimate repetition count and completion time, subject to a safe setup and appropriate screening.

Output: Completion time, repetition count and assessment-quality metadata. It is a functional measure, not a diagnosis.

How the future data would join the existing system

Guided phone session Patient starts a test	**On-device processing** Signal / pose + quality checks	**Structured observation** Value, unit, time, source, quality	**Existing VitalWatch pipeline** Temporal model → evidence → RAG → privacy gate → LLM

Integration principle: Each phone-derived result becomes a timestamped, source-labelled observation in the same general data contract used for other patient measurements. The existing temporal model and reasoning pipeline remain the core; a new input adapter supplies the additional observations.

Data, validation and privacy requirements

Data contract	Validation before any clinical use	Privacy safeguards
Observation type; value and unit; timestamp; source/device; signal or test quality; consent status; evidence ID. Preserve whether a value is measured, extracted, patient-reported or simulated.	Compare camera pulse estimates with an appropriate reference such as ECG; compare mobility counts/timing with trained-observer or reference measurements. Test repeatability, device variation, poor-signal rejection and usability in the intended population.	Obtain explicit consent; process video/audio locally where possible; avoid retaining raw media by default; minimize identifiers; encrypt stored data; apply role-based access and audit logging. Never send raw media to an external LLM by default.

Possible later expansion

After the smartphone workflow is independently tested, VitalWatch may add opt-in integrations with compatible wearables and external devices (for example, validated blood-pressure monitors or pulse oximeters). These sources should use the same timestamped observation structure and must retain device/source provenance. Device integration is a future extension, not a dependency of the current project.

Current MVP boundary: Smartphone sensing, wearables and external-device integration are explicitly excluded from the present implementation and primary experiments. VitalWatch v1.0 continues to use the defined clinical dataset, synthetic chronological replay, temporal risk model, temporal evidence, Patient RAG, privacy/evidence firewall and constrained LLM reasoning. Any future sensor-derived model use requires separate technical and clinical validation.

VitalWatch — Detailed Technical Revision

Expanded addendum to the original pitch. The original proposal is preserved in full; the pages that follow deepen the architecture, research method, privacy design, fine-tuning plan, orchestration workflow, and implementation roadmap.

What is being added

The original pitch establishes the early-warning foundation: chronological patient observations, causal trend features, patient-level evaluation, a replayable interface, and a human-review safety boundary. This revision retains that foundation and adds a longitudinal evidence and explanation layer around it.

Original foundation (retained)	Detailed additions in this revision
Historical hourly replay; multivariate vital trends; missingness indicators; threshold baseline; interpretable ML; patient-level splits; FastAPI and dashboard.	Durable patient records + Redis context cache; patient-scoped RAG; fine-tuned local SLM; privacy validation; external LLM explanation; Agent Orchestrator verification; expanded temporal-model experiments.

Updated project statement

VitalWatch is a longitudinal healthcare intelligence research prototype that organizes patient observations over time, forecasts selected clinical measurements or deterioration risk, retrieves relevant historical evidence, and produces evidence-constrained explanations for clinician review.

The temporal model remains the predictive authority. RAG supplies relevant context. The internal SLM structures and minimizes evidence before any optional external transmission. The external LLM is the reasoning and explanation layer constrained by model outputs and cited evidence. The orchestrator checks the generated response before it reaches the dashboard.

Important scope clarification

The original pitch's PhysioNet/CinC Challenge 2019 sepsis early-warning task remains a defensible benchmark for the first deterioration-risk experiment. The broader longitudinal report/RAG workflow is an architectural extension and should be validated separately. Do not imply that one dataset validates every VitalWatch capability.

1. Updated System Architecture

The system is divided into a trusted patient-data plane, a temporal prediction plane, and a controlled explanation plane. These paths meet only when a minimized evidence package is assembled.

INPUTS Vitals · labs · history PDF reports · synthetic cases	**INGESTION** OCR / extraction Normalize · validate Timestamp · provenance	**PERMANENT STORE** PostgreSQL Source of truth Versioned observations
CONTEXT CACHE Redis Recent summaries Session / conversation state	**TEMPORAL ENGINE** Features · time gaps Missingness · trends Forecast / risk models	**RAG INDEX** Patient-scoped vector store Source-linked chunks Access-controlled retrieval
RETRIEVAL Task-conditioned evidence Relevant reports / history Source references	**LOCAL SLM** Fine-tuned evidence compiler Structured, minimized context No diagnosis authority	**PRIVACY GATE** PII / leakage checks Clinical fact preservation Schema + provenance checks
PACKAGE BUILDER Approved evidence + temporal outputs + uncertainty	**EXTERNAL LLM** Constrained explanation Differential possibilities Evidence and uncertainty	**AGENT ORCHESTRATOR** Claim / number / time checks Contradiction checks Retry once or abstain
DASHBOARD Timeline · trends · forecasts Evidence references · alerts	**CLINICIAN REVIEW** Human interpretation Acknowledgement / feedback	**AUDIT** Access and transmission events Model / prompt versions No unnecessary raw PHI

Data flow: all incoming records are normalized and validated before they update the durable record. Temporal features and forecasts are computed from timestamped observations available up to the prediction time. Patient RAG retrieves only task-relevant evidence. The local SLM converts retrieved material into a structured evidence package; deterministic validators and privacy checks decide what may leave the local boundary. The external LLM explains the approved evidence, and the orchestrator verifies the response.

Key architectural rules

- PostgreSQL (or an equivalent durable database) is the source of truth. Redis is a cache, not the medical record store.
- The prediction pipeline must not depend on an LLM response. Predictions are computed independently and remain inspectable.
- Raw patient documents and raw RAG chunks stay inside the controlled data boundary by default.
- Every extracted observation retains source ID, timestamp, unit, extraction confidence, and validation status.
- The external LLM receives only the minimum approved evidence needed for the current task; external reasoning can be disabled.
- If evidence or privacy validation fails, the system abstains rather than silently sending raw text.

2. Data Ingestion, Patient Context & Temporal Modelling

This layer turns heterogeneous records into a consistent, auditable longitudinal timeline. It must preserve time, units, source provenance, and missingness.

2.1 Report ingestion and normalization

Stage	Implementation detail	Required output / control
Upload	Accept PDF, image, CSV and structured observations. Validate file type and size; authenticate the patient/session.	Upload record, owner, timestamp, checksum, access decision.
Document extraction	OCR for scanned reports; text and table extraction for digital PDFs. Retain page and region references where available.	Raw extracted text/table plus source document ID.
Structured extraction	Use rules and/or the local SLM to identify parameter, value, unit, collection date, reference range and source context.	Typed observation candidates; never treat free text as validated facts.
Validation	Check schema, date plausibility, unit conversion, duplicates, impossible values, conflicting results and missing fields.	Accepted / rejected / needs-review status with reason.
Persistence	Write validated observations to durable storage and index source-linked text for retrieval.	Versioned timeline + vector index with provenance.

Example normalized observation

```
{"patient_id":"P001","parameter":"hemoglobin","value":11.2,
"unit":"g/dL","timestamp":"2026-10-15T09:30:00",
"source_id":"REPORT_001","confidence":0.96,
"validation_status":"accepted"}
```

Confidence is an extraction signal, not proof of correctness. Store the original text span or source location so a reviewer can inspect where the value came from.

2.2 Redis context cache

Redis is used to make recent patient context fast to retrieve across dashboard requests and multi-step reasoning sessions. It must never become the only copy of a medical record or prediction.

Example key (illustrative)	Cached value	Freshness / invalidation
patient:P001:recent_observations	Recent validated vitals and labs	Short TTL; invalidate when new observations are committed.
patient:P001:trend_summary	Latest causal trend summary and missingness state	Recompute when underlying observation version changes.
patient:P001:recent_predictions	Prediction value, horizon, model version, timestamp	Expire or mark stale on new data or model-version change.
patient:P001:conversation_context	Recent conversation/task state, not raw full chart	Session TTL; strict patient and user isolation.
patient:P001:profile_summary	Minimum necessary profile context	Versioned; refresh after profile update.

Security requirements: patient-scoped key namespaces; authorization before cache access; TLS in transit; encryption at rest where supported; least-privilege service accounts; TTL and explicit invalidation; no sensitive payloads in application logs; stale prediction rejection; and cache-miss fallback to the durable database.

2.3 Temporal model strategy

Start with a small number of well-defined prediction tasks. For each task, define the prediction time, input window, forecast horizon, target, unit, and acceptable missingness before selecting a model.

Task	Example target	Candidate models
Vital-value forecasting	Next-hour / next-day HR, BP, glucose or lab value	Persistence + linear baseline, XGBoost, LSTM, GRU, time-aware GRU/LSTM
Deterioration risk	Event within a fixed future window	Logistic regression, XGBoost, GRU-D, time-aware sequence model
Abnormality detection	Unexpected deviation from a patient's recent pattern	Robust statistical baseline, Isolation Forest, autoencoder
Multi-parameter trend	Joint future trajectory for selected measurements	Independent models first; compare shared multi-output model later

Missingness is informative but must be handled carefully. Include masks and elapsed-time-since-last-measurement features; compare simple imputation against missingness-aware approaches such as GRU-D. Do not treat an unmeasured value as normal.

3. Research Methodology & Model Evaluation

The research contribution should be demonstrated through controlled comparisons and ablation studies, not by claiming a novel architecture without evidence.

3.1 Experimental questions

- Does explicit elapsed-time information improve forecasting compared with an ordinary LSTM or GRU?
- Do missingness masks and time-since-last-observation features improve performance over conventional imputation?
- Do patient-specific baselines and causal trend features improve over current-value-only models?
- Does a shared multi-output model outperform independent per-parameter models, or does it introduce negative transfer?
- Can uncertainty estimates identify forecasts that should be withheld from the explanation layer?
- How much do OCR and extraction errors degrade downstream temporal prediction?

3.2 Baseline and comparison matrix

Experiment	Model / condition	Purpose
E0	Persistence / last observation carried forward	Simple forecasting reference.
E1	Linear or logistic regression with causal engineered features	Interpretable baseline.
E2	XGBoost / gradient-boosted trees	Strong tabular ML comparator.
E3	LSTM	Sequence-learning baseline.
E4	GRU	Lower-complexity recurrent comparator.
E5	GRU-D or mask + elapsed-time model	Missingness-aware sequence modelling.
E6	Time-aware LSTM/GRU with patient baseline features	Test whether irregular timing adds value.
E7	Attention-based sequence model or shared multi-output model	Optional extension only if data volume supports it.

3.3 Evaluation protocol

- Split by patient, not by hourly row. Keep each patient's full timeline in only one split.
- Use chronological ordering within each patient. At prediction time t, features may use only observations at or before t.
- Fit imputation, scaling, feature selection, calibration and thresholds on training/validation data only.
- Tune alert thresholds on validation patients; report final results once on held-out test patients.
- Report subgroup and missingness-stratified performance where sample sizes permit.
- For the PhysioNet/CinC 2019 task, state the operational label definition and prediction horizon explicitly; do not describe the label as exact biological onset.

Problem type	Metrics to report
Regression / forecasting	MAE, RMSE, R^2; error by forecast horizon; prediction interval coverage if uncertainty is implemented.
Binary event prediction	AUROC, AUPRC, sensitivity, specificity, precision, recall, F1, calibration, false alerts per patient-day.
Early-warning utility	Lead time, sensitivity at a fixed false-alert burden, alert persistence, time under alert.
Data extraction	Field-level precision/recall/F1, unit accuracy, date accuracy, source traceability, reviewer correction rate.

3.4 Ablations and robustness

- Remove missingness features to measure their contribution.
- Remove rolling trends and patient-specific baselines to test whether trajectory information helps.
- Compare fixed imputation, time-aware imputation and GRU-D-style handling.
- Evaluate with synthetic missingness and naturally missing records separately.
- Perturb extracted values, units and dates to quantify ingestion-error propagation.
- Report uncertainty and failure cases, not only aggregate accuracy.

Synthetic data is useful for controlled scenarios, demo replay and edge-case testing. It is not a substitute for external clinical validation and must be clearly labeled in all outputs.

4. Internal SLM: Fine-Tuning Scope & Dataset Plan

The internal small language model is a local evidence compiler. It is not the temporal predictor, not the external-facing chatbot, and not an autonomous diagnostic model.

4.1 Exact responsibility

- Convert relevant retrieved report snippets into a strict structured representation.
- Preserve clinically meaningful values, units, timestamps or relative-time intervals, negation, uncertainty and source references.
- Remove irrelevant narrative and minimize identifiers only when doing so does not destroy clinically necessary context.
- Flag ambiguous, conflicting or incomplete evidence for deterministic validation or human review.
- Return schema-valid JSON and abstain when a field cannot be supported by the source.

4.2 Fine-tuning dataset assignment

Dataset / source	Role in fine-tuning	Access and use condition
ELMTEX	Clinical entity / information extraction examples; use for mapping text into structured clinical concepts if its annotations and license fit the task.	Verify dataset card, license, splits and annotation schema before use.
Synthetic instruction examples	Primary controllable source for VitalWatch-specific JSON schema, relative-time conversion, unit preservation, conflict handling, abstention and minimization.	Generate from templates and clinically reviewed rules; label as synthetic.
MIMIC-IV-Ext-Instr	Potential instruction-following / clinical task examples to broaden instruction format.	Use only after access approval and license/DUA review; do not assume public unrestricted access.
n2c2 / i2b2 de-identification tasks	Potential PII detection and de-identification examples; useful for evaluation and possibly task-specific training.	Access may require registration or agreements; normalize annotation formats before combining.

Start with ELMTEX plus a carefully designed synthetic instruction set. Add restricted datasets only after confirming access, licensing, annotation compatibility and permitted use. Keep dataset provenance and original split identifiers; do not merge incompatible labels blindly.

4.3 Candidate base models and training approach

Candidate	Why benchmark it	Approach
Qwen2.5-1.5B-Instruct	Compact instruction model candidate.	4-bit QLoRA / LoRA adapter.
Llama 3.2 3B Instruct	Larger small-model comparator; review license and hardware needs.	4-bit QLoRA / LoRA adapter.
Gemma 2 2B IT	Alternative compact instruction model.	4-bit QLoRA / LoRA adapter.

Do not train a language model from scratch. Benchmark at least one base model before fine-tuning, then compare the fine-tuned adapter against the unmodified model and a deterministic extraction baseline. A 1B–3B parameter model is a starting range, not a guaranteed fit; actual GPU memory, latency and quality must be measured.

4.4 Training data format and controls

Use instruction/response records with source text, task instruction, expected structured JSON, provenance and synthetic/real source labels. Split by document or patient so near-duplicate snippets cannot leak across train, validation and test. Include edge cases: conflicting dates, duplicate tests, missing units, negation, uncertain language, impossible values, multiple measurements, relative time, OCR noise, prompt injection and insufficient evidence.

4.5 Evaluation

Evaluation area	Measure
Extraction quality	Field-level precision, recall and F1; exact match for values, units and dates.
Clinical fidelity	Negation, temporality, uncertainty, source attribution and numerical preservation.
Privacy	Direct identifier leakage, indirect disclosure risk, sensitive-attribute leakage and minimum-disclosure tests.
Robustness	Malformed JSON, prompt injection in retrieved notes, conflicting evidence and abstention behaviour.
Operations	Latency, throughput, GPU memory, model size and failure rate.

5. Privacy Gate, External LLM & Agent Orchestrator

Privacy and hallucination control are implemented as separate, testable controls. Fine-tuning alone does not anonymize data; an orchestrator alone does not eliminate hallucinations.

5.1 Privacy-preserving evidence pipeline

Step	Action	Fail-safe
1. Authorization	Confirm user access to the patient and task before retrieval.	Deny retrieval if authorization fails.
2. Local retrieval	Search only the patient-scoped local vector index; retrieve the minimum relevant chunks.	Reject cross-patient or unscoped retrieval.
3. Local SLM	Transform chunks into structured evidence with source IDs, values, units, time and uncertainty.	Mark unsupported fields unknown; do not infer.
4. Deterministic checks	Validate schema, dates, units, source IDs, contradictions and numerical fidelity.	Quarantine invalid evidence for review.
5. Privacy validation	PII/identifier scan, quasi-identifier review, leakage checks and minimization policy.	Block external transmission if policy fails.
6. Approved package	Create task-specific minimal evidence package; record what is transmitted.	Allow local-only mode if external reasoning is disabled.
7. External LLM	Generate constrained explanation from approved package only.	No raw report upload or full-chart prompt.
8. Audit	Record access, model/prompt versions and transmission metadata without unnecessary raw PHI.	Reviewable audit trail.

Example minimized evidence

```
{"subject_reference":"SUBJECT_001",
"observations":[{"parameter":"haemoglobin","value":12.8,
"unit":"g/dL","relative_time":"T1",
"source_reference":"REPORT_001"}]}
```

Relative time can reduce exposure while preserving intervals, but removing names or exact dates does not guarantee anonymity. Rare diagnoses, precise values, combinations of events and narrative details may still identify a person. Use privacy-conscious / minimized language, not claims of perfect anonymization.

5.2 Privacy experiment

Variant	Pipeline	Question
P1	RAG chunks sent directly to external LLM	Baseline disclosure exposure.
P2	RAG → rule-based minimization / PII filtering → external LLM	Does deterministic filtering reduce leakage while preserving facts?
P3	RAG → fine-tuned local SLM → deterministic validator + privacy gate → external LLM	Does the learned evidence compiler improve structure and minimum disclosure?

Compare PII leakage, sensitive-information leakage, preservation of clinical facts, value/unit/time consistency, source traceability, latency, token use and explanation quality. Include a local-only condition as a deployment fallback.

5.3 Agent Orchestrator verification

Verifier	What it checks	Preferred method
Evidence verifier	Every material claim maps to supplied evidence IDs; unsupported claims are removed or qualified.	Deterministic evidence-ID matching plus semantic entailment review.

Verifier	What it checks	Preferred method
Prediction verifier	Forecast values, units, horizon, timestamps, risk score and model version match the prediction service output.	Deterministic Python checks.
Temporal / contradiction verifier	No future data, reversed chronology, unsupported causal claim or contradiction with supplied evidence.	Timestamp rules + contradiction classifier / NLI as a secondary signal.
Safety / schema verifier	Valid output structure, uncertainty language, no authoritative diagnosis or treatment instruction.	Pydantic schema + policy rules.
Revision controller	Provides structured correction feedback and allows at most one retry.	Bounded state machine; abstain if still invalid.

The orchestrator should be a bounded verification workflow, not a free-running multi-agent medical debate. Use deterministic checks for numbers, units, dates, IDs and schemas. Semantic/NLI checks may supplement these rules but should not be the sole judge. If verification still fails after one revision, return an explicit insufficient-evidence or unable-to-verify response.

6. Explanation Contract, Backend Services & Dashboard

The explanation must be auditable and visibly separate observed facts, model predictions and hypotheses.

6.1 External LLM output contract

Field	Requirement
Summary	Short description of the temporal pattern; no diagnostic certainty.
Observed evidence	Timestamped measurements, direction of change, persistence and source references.
Model output	Prediction target, value/risk, forecast horizon, model version and uncertainty.
Possible interpretations	Clearly labelled hypotheses or differential possibilities, not confirmed diagnoses.
Evidence against / contradictions	Relevant observations that weaken or conflict with each interpretation.
Missing information	Important unavailable measurements, gaps or context.
Uncertainty	Explicit confidence/limitations; distinguish model probability from clinical certainty.
Traceability	Each material claim links to evidence IDs or prediction output IDs.
Safety note	For clinician review only; no autonomous diagnosis, prescription or escalation.

6.2 Suggested backend services

Service	Responsibility
Patient / Access API	Authentication, patient authorization and role-aware access.
Ingestion API	Upload, OCR/extraction jobs, validation and durable writes.
Temporal Feature Service	Causal feature calculation and missingness/time-gap handling.
Prediction Service	Model inference, horizon metadata, uncertainty and model versioning.
RAG Service	Patient-scoped vector retrieval with source provenance and relevance filtering.
SLM Evidence Compiler	Local fine-tuned model inference and strict structured output.
Privacy Gate	PII detection, minimization, leakage rules and external-transmission decision.
Reasoning Adapter	External LLM call with approved evidence and constrained prompt.
Orchestrator	Evidence, prediction, temporal and safety verification; bounded retry / abstention.
Dashboard / Replay API	Timeline, charts, replay controls, explanations and review status.

6.3 Dashboard additions

- Patient timeline with observations, report events, missing-data intervals and source references.
- Trend charts showing raw observations, causal rolling summaries, forecast intervals and model confidence.
- Evidence panel separating observed facts, predictions and LLM-generated hypotheses.
- RAG trace panel showing which source records were retrieved and why, with privacy-safe display.
- Verification status: passed, revised, abstained, or local-only; do not hide failed checks.
- Model card / run metadata: model version, training cohort, target, horizon, evaluation split and limitations.
- Simulation and synthetic-data labels visible throughout replay and screenshots.

6.4 API sketch

```
POST /patients/{id}/reports
POST /patients/{id}/observations
GET /patients/{id}/timeline
GET /patients/{id}/trends?parameter=heart_rate
POST /patients/{id}/predictions
POST /patients/{id}/evidence/explain
GET /patients/{id}/explanations/{explanation_id}
WS /replay/{session_id}
```

API responses should include timestamps, units, source IDs, model versions, forecast horizon and validation state. Avoid returning raw patient context to clients that do not need it.

7. Detailed Implementation Plan & Delivery Gates

The implementation order is designed to keep the predictive core testable before adding the LLM layer. Each phase has a concrete exit condition.

Phase	Work items	Exit gate
0. Scope and contracts	Freeze MVP prediction target/horizon; define observation schema, units, patient split strategy, safety language and synthetic scenarios.	Approved PRD, schema, evaluation protocol and repository structure.
1. Data pipeline	Load PhysioNet benchmark; parse reports; implement OCR/extraction interface; normalize dates/units; provenance and validation.	Golden test cases pass; invalid/conflicting values are flagged.
2. Temporal baseline	Build causal rolling features, missingness masks, elapsed-time features, threshold baseline and logistic/XGBoost model.	Patient-level baseline metrics reproducible; no future leakage.
3. Sequence experiments	Train LSTM/GRU and missingness-aware/time-aware variants; compare ablations and uncertainty.	Experiment table, held-out evaluation and failure analysis.
4. Durable store + Redis	Persist patient timeline in DB; add patient-scoped Redis cache, TTL, invalidation and stale-prediction checks.	Cache cannot override source of truth; access-isolation tests pass.
5. RAG	Chunk reports with source IDs; create patient-scoped vector index; task-conditioned retrieval; prompt-injection tests.	Retrieval is access-controlled, relevant and traceable.
6. SLM fine-tuning	Prepare approved datasets and synthetic instructions; benchmark base models; QLoRA fine-tune; evaluate extraction and leakage.	Adapter, dataset manifest, training report and reproducible inference.
7. Privacy gate	Add PII/identifier detection, minimization, deterministic clinical-fact validation and external-call allow/deny policy.	P1/P2/P3 privacy comparison completed; local-only mode works.
8. LLM + orchestrator	Implement structured prompt/output contract; evidence/prediction/temporal/safety verifiers; one retry then abstain.	Unsupported claims and mismatched numbers are caught in tests.
9. UI + integrated demo	Timeline, trends, forecast view, evidence panel, verification state, synthetic replay and clinician review flow.	Stable end-to-end demo with limitations visible.

Recommended build order

Do not build the SLM and agent system first. First establish a validated observation timeline and an independent prediction baseline. Then add retrieval and evidence compilation. Finally add external reasoning and verification. This prevents LLM-generated text from masking weak data quality or an unvalidated prediction model.

MVP / later boundary

MVP	Later development
Upload reports; synthetic data generator; structured observations; durable DB + Redis cache; patient timeline; selected temporal forecasts / deterioration risk; patient RAG; local SLM evidence compiler; privacy gate; optional external LLM; orchestrator; clinician-facing dashboard.	Wearable integrations; smartphone camera-based pulse estimation; pose/mobility tracking; accelerometer/gyroscope ingestion; voice/respiratory monitoring; hospital EHR integration; prospective clinical deployment; additional outcomes after separate validation.

8. Risks, Limitations, Governance & References

VitalWatch is a research prototype. The following risks must be explicit in the pitch, demo and implementation report.

Risk	Mitigation / honest claim
Dataset shift and limited generalization	Use patient-level splits, external validation where possible, and avoid claims that a single benchmark generalizes to all hospitals.
Label limitations	Define the target operationally; retrospective labels are not necessarily exact onset times or a clinical diagnosis.
Missing and irregular measurements	Model masks and elapsed time; report performance by missingness; do not interpret unmeasured as normal.
Extraction/OCR errors	Preserve source evidence, validate units/dates, flag conflicts and allow human correction.
Stale context in Redis	TTL, versioning, invalidation, database fallback and stale-prediction rejection.
Privacy leakage	Local-first storage, minimization, PII checks, quasi-identifier review, transmission controls and privacy testing.
LLM hallucination	Evidence-linked output, deterministic checks, bounded retry, explicit abstention and clinician review.
Automation bias	Separate facts/predictions/hypotheses visually; never present generated interpretation as confirmed diagnosis.
Security and access control	Patient-scoped authorization, least privilege, encryption, audit trail and synthetic demo data.

Research and technical references

- PhysioNet / Computing in Cardiology Challenge 2019 — Early Prediction of Sepsis from Clinical Data: physionet.org/content/challenge-2019.
- PhysioNet data resources: physionet.org.
- FastAPI documentation and WebSockets: fastapi.tiangolo.com.
- scikit-learn documentation: scikit-learn.org.
- Hugging Face Transformers, Datasets and PEFT documentation: huggingface.co/docs.
- Microsoft Presidio for PII detection and anonymization: microsoft.github.io/presidio.
- Redis documentation: redis.io/docs.

Final architecture principle

Validated records → temporal features and predictions → task-conditioned patient RAG → local SLM evidence compilation → deterministic privacy gate → constrained external reasoning → orchestrator verification → clinician review.

Safety boundary: VitalWatch prioritizes review and organizes evidence. It does not independently diagnose, prescribe treatment, or autonomously escalate care. Clinical use would require prospective evaluation, external validation, governance approval, privacy/security review and workflow-specific testing.